

M1 JAVA WEB 1

1. TUJUAN

CPMK: Mahasiswa mengetahui dan memahami dasar dari web dan dasar microservices.

Sub-CPMK:

- a. Mahasiswa mampu mengimplementasikan REST Api sederhana dengan menggunakan bahasa Java Spring

2. DURASI WAKTU

1 pertemuan x 3 jam

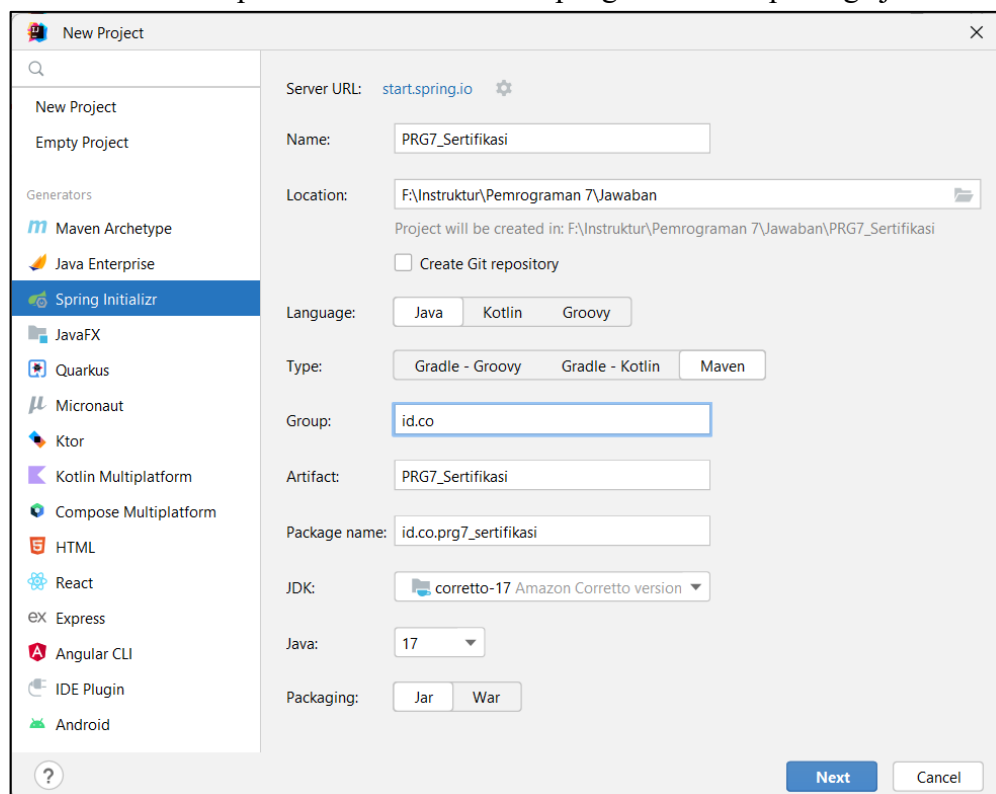
3. DASAR TEORI

Java Web

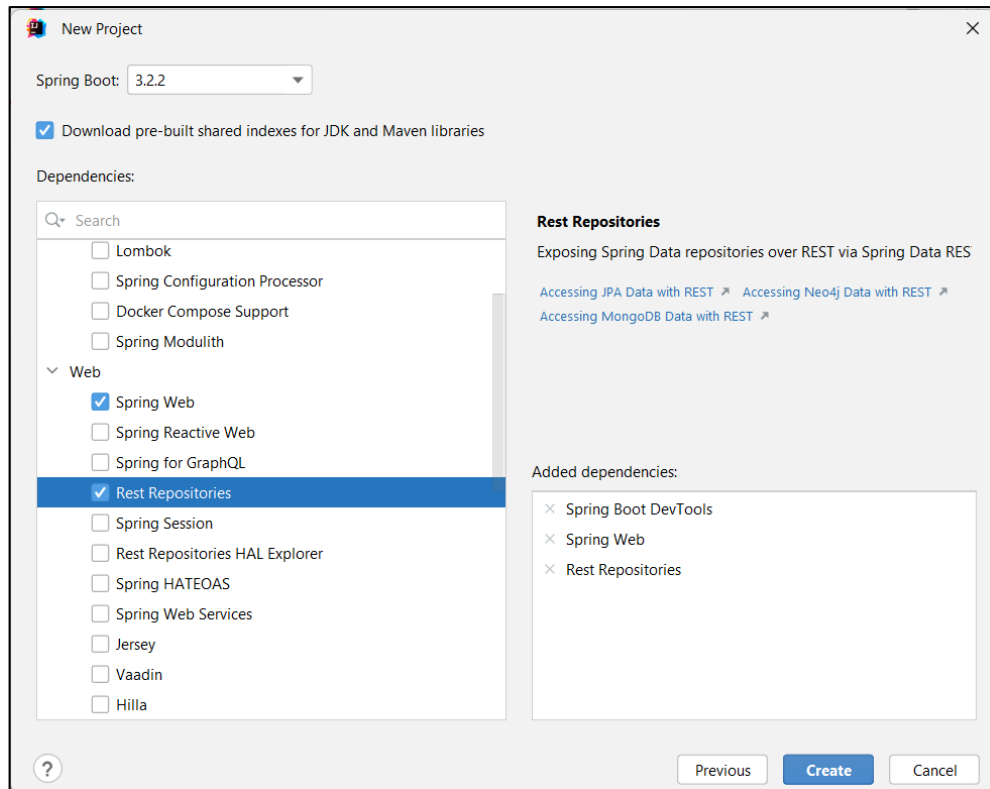
4. PERCOBAAN

A. Membuat Aplikasi Spring

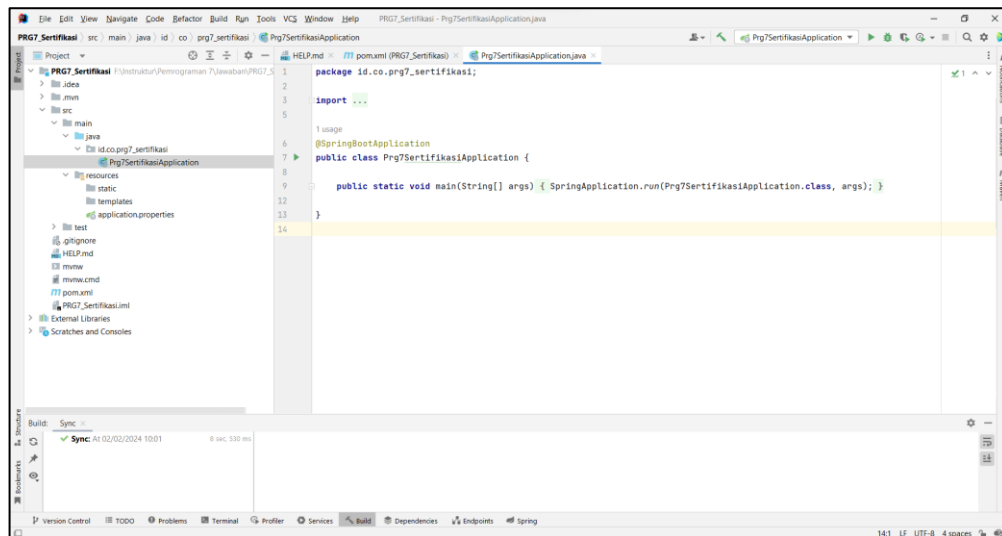
1. Buatlah proyek baru dengan nama PRG7_Sertifikasi_XXX dimana XXX adalah 3 angka paling belakang NIM anda. Click **File > New > Project > Spring Initializer > Next**. Nama Group dan Artifact akan mempengaruhi nama package java anda.



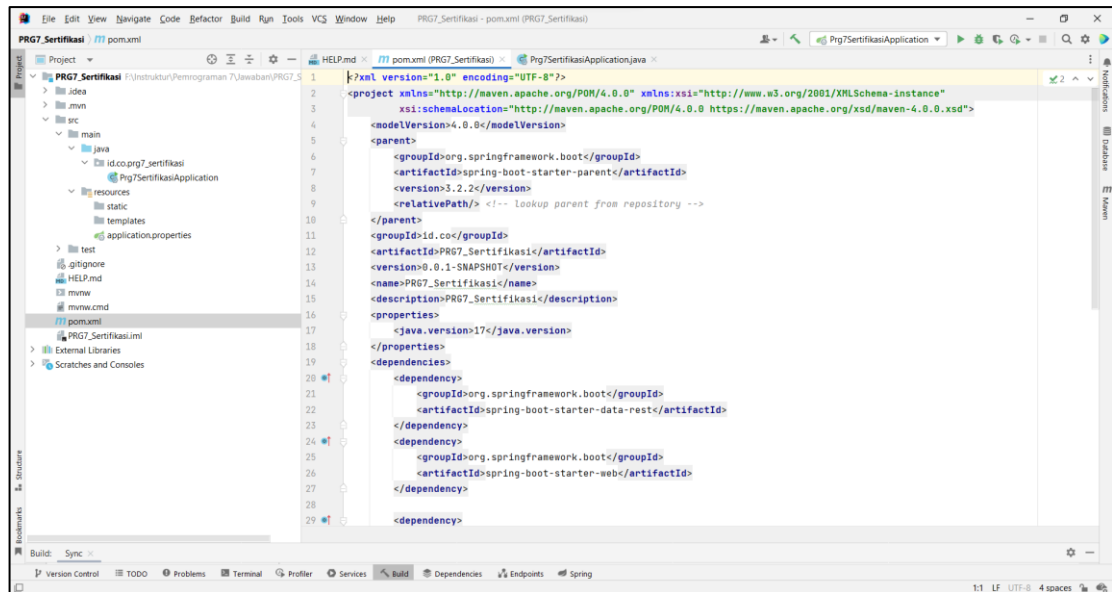
2. Penambahan dependencies untuk mendukung pembuatan aplikasi Spring, saat ini kita menambahkan Spring Web dan Rest Repositories. Dependencies dapat ditambahkan sesuai dengan kebutuhan anda.



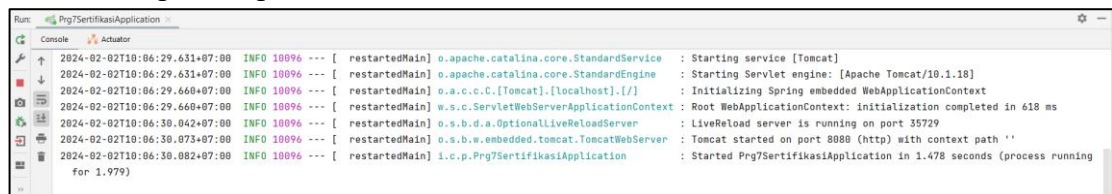
3. Klik create, maka IDE akan membuat proyek kita, dan akan terdapat beberapa file dan package seperti dibawah ini



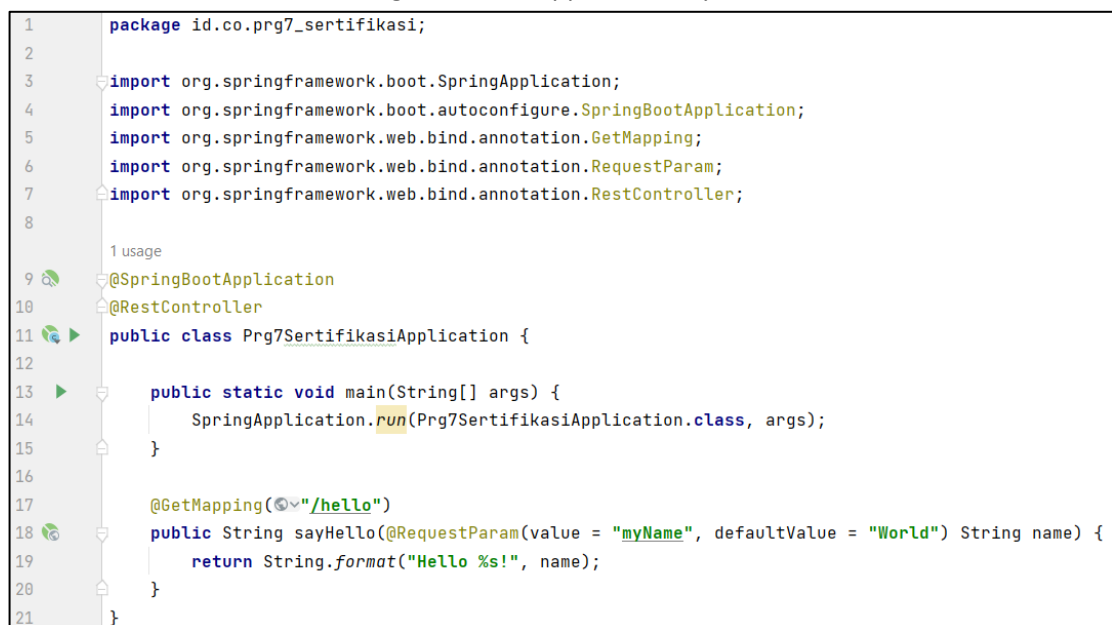
4. Bukalah file pom.xml, disini kita dapat mengkonfigurasi aplikasi java web yang kita buat, seperti konfigurasi versi java dan dependencies yang kita gunakan



5. Coba jalankan aplikasi yang sudah anda buat, jika tidak ada kendala maka akan muncul tampilan seperti berikut.

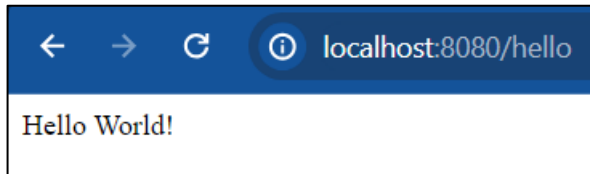


6. Tambahkan kode didalam file Prg7SertifikasiApplication seperti dibawah ini.

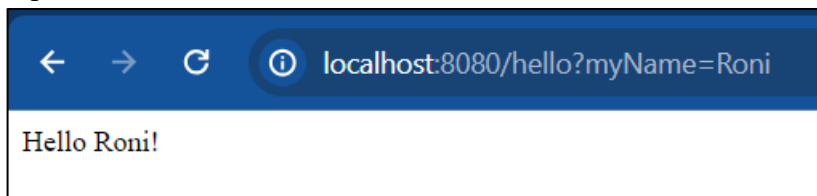


- a. Metode sayHello() mengambil parameter nama dan mengembalikan kata Hello yang digabungkan dengan nilai parameter. Semua hal lainnya diatur dengan menambahkan anotasi Spring:
- b. Anotasi @RestController menandai kelas SpringBootTutorialApplication sebagai penangan permintaan (REST controller).
- c. Anotasi @GetMapping("/hello") memetakan metode sayHello() ke permintaan GET untuk /hello.

- d. Anotasi `@RequestParam` memetakan parameter metode nama ke parameter permintaan web `myName`. Jika Anda tidak memberikan parameter `myName` dalam permintaan web Anda, nilainya akan default ke `World`.
7. Coba jalankan kembali aplikasi tersebut, lalu akses <http://localhost:8080/hello>. Maka akan muncul tampilan seperti dibawah ini



8. Apabila kita mengakses dengan menggunakan parameter `myName` maka akan tampil seperti dibawah ini!



9. Selanjutnya, buatlah package dengan nama `model` dan `rest`, disini kita akan membuat API secara sederhana, didalam package `model`, buatlah class dengan nama `User`

```
1 package id.co.prg7_sertifikasi.model;
2
3 public class User {
4     3 usages
5     private int id;
6     3 usages
7     private String nama;
8     3 usages
9     private String username;
10    3 usages
11    private String password;
12    3 usages
13    private String email;
14    3 usages
15    private String role;
16    3 usages
17    private int status;
18
19    public User() {
20    }
21
22    4 usages
23    public User(int id, String nama, String username, String password, String email, String role, int status) {...}
24
25    //GETTER SETTER
26    3 usages
27    public int getId() {
28        return id;
29    }
```

10. Selanjutnya, buatlah kelas `UserRest` di dalam package `rest`, disini kita akan membuat variabel secara static dengan nama `users`

```

1 package id.co.prg7_sertifikasi.rest;
2
3 import id.co.prg7_sertifikasi.model.User;
4 import org.springframework.web.bind.annotation.*;
5
6 import java.util.ArrayList;
7 import java.util.List;
8
9 @RestController
10 @RequestMapping("/users")
11 public class UserRest {
12     public static List<User> users;
13
14     static {
15         users = new ArrayList<>();
16         users.add(new User(1, "Roni Prasetyo", "roni.p", "password123", "roni@gmail.com",
17             "Admin", 1));
18         users.add(new User(2, "Sisia Dika", "sisia.dika", "password456", "sisia@gmail.com",
19             "Admin", 1));
20         users.add(new User(3, "Ning Ratwastuti", "ning.r", "password789", "ning@gmail.com",
21             "Admin", 1));
22         users.add(new User(4, "Elora Manuella", "elora.a", "password101", "elora@gmail.com",
23             "Admin", 1));
24     }
25 }

```

11. Selanjutnya, buatlah method dengan nama getUsers, lalu gunakan juga anotasi @GetMapping

```

24 @GetMapping("/getUsers")
25 public List<User> getUsers() {
26     return users;
27 }

```

12. Coba jalankan aplikasinya, lalu buka linknya

localhost:8080/users/getUsers

```

[{"id":1,"nama":"Roni Prasetyo","username":"roni.p","password":"password123","email":"roni@gmail.com","role":"Admin","status":1}, {"id":2,"nama":"Sisia Dika","username":"sisia.dika","password":"password456","email":"sisia@gmail.com","role":"Admin","status":1}, {"id":3,"nama":"Ning Ratwastuti","username":"ning.r","password":"password789","email":"ning@gmail.com","role":"Admin","status":1}, {"id":4,"nama":"Elora Manuella","username":"elora.a","password":"password101","email":"elora@gmail.com","role":"Admin","status":1}]

```

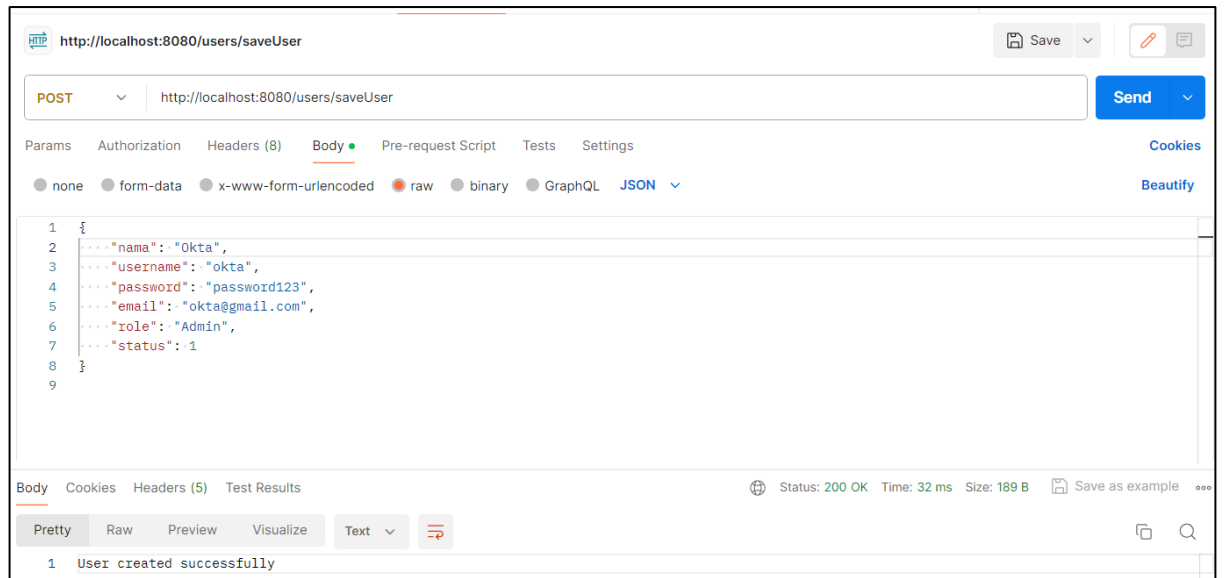
13. Selanjutnya, buatlah untuk PostMapping dengan nama method createUser

```

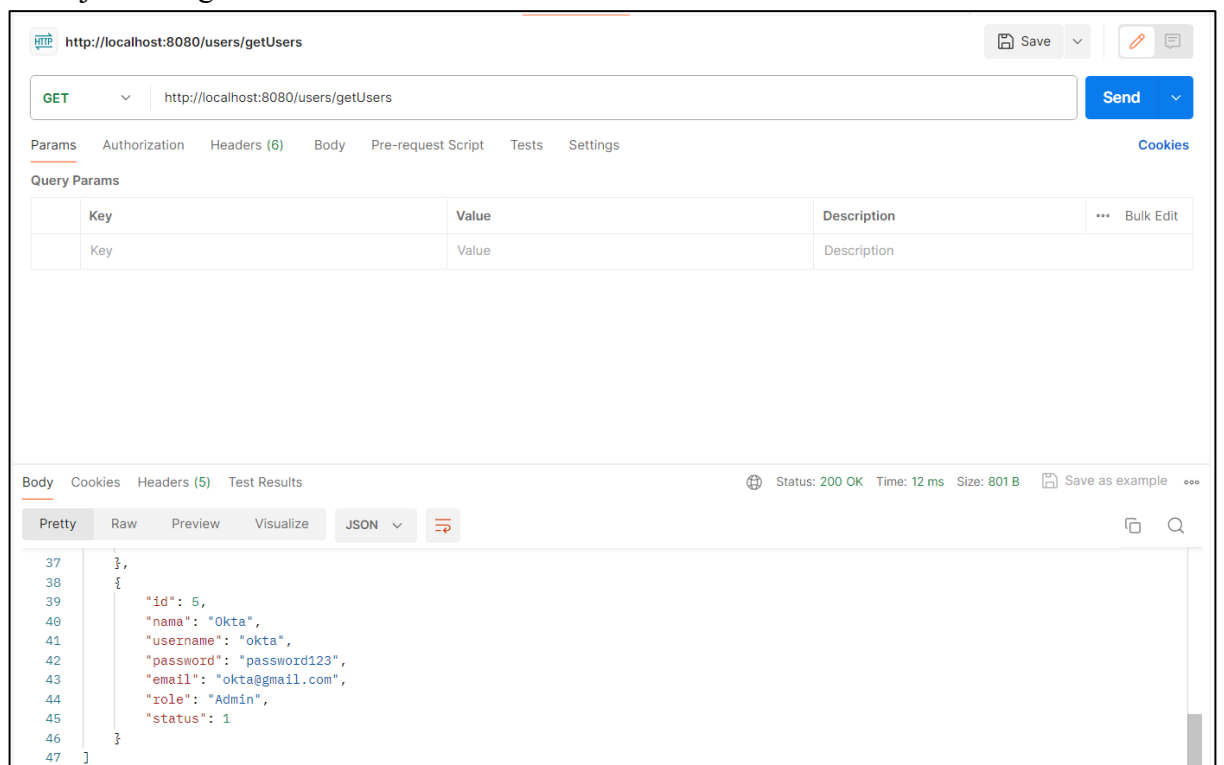
29 @PostMapping("/saveUser")
30 public String createUser(@RequestBody User user) {
31     int nextId = users.size() + 1;
32     user.setId(nextId);
33     users.add(user);
34     return "User created successfully";
35 }

```

14. Coba jalankan di dalam postman seperti dibawah ini



15. Coba jalankan `getUsers` maka data akan bertambah 1



16. Selanjutnya, tambahkan untuk `getUserById` seperti dibawah ini, ini akan memfilter data statis yang kita punya

```

35 @GetMapping("/getUser/{id}")
36 public User getUserById(@PathVariable int id) {
37     for (User user : users) {
38         if (user.getId() == id) {
39             return user;
40         }
41     }
42     return null;
43 }

```

17. Jalankan aplikasinya lalu cobalah menggunakan API ini

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/users/getUser/1`. The response status is 200 OK, and the response body is a JSON object:

```

{
  "id": 1,
  "name": "Roni Prasetyo",
  "username": "roni.p",
  "password": "password123",
  "email": "roni@gmail.com",
  "role": "Admin",
  "status": 1
}

```

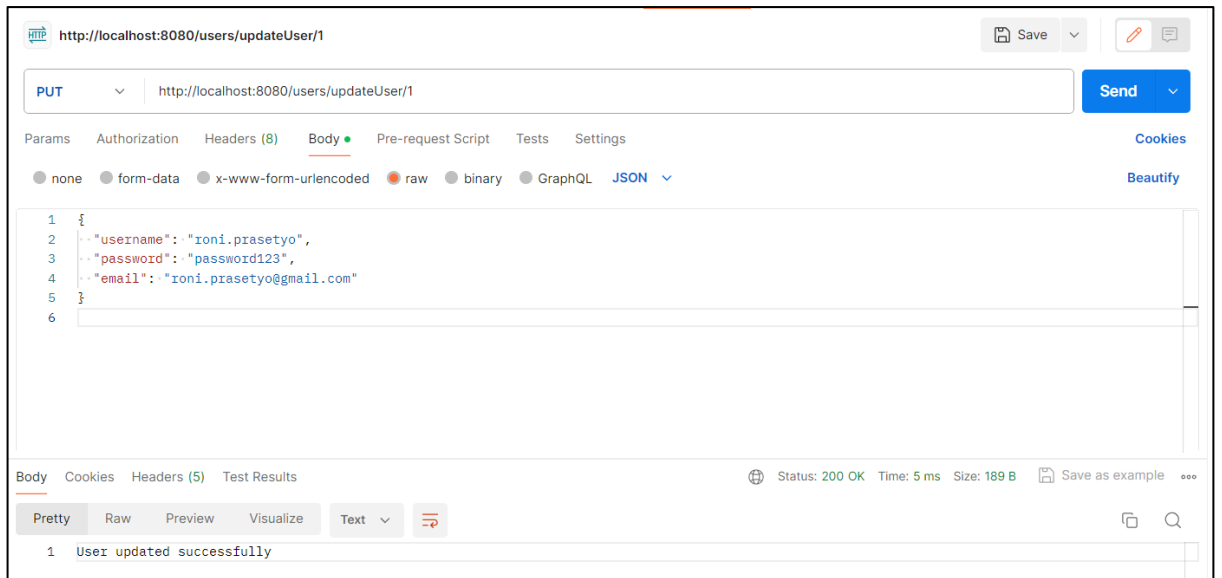
18. Tambahkan juga untuk update dimana akan menggunakan anotasi `@PutMapping`

```

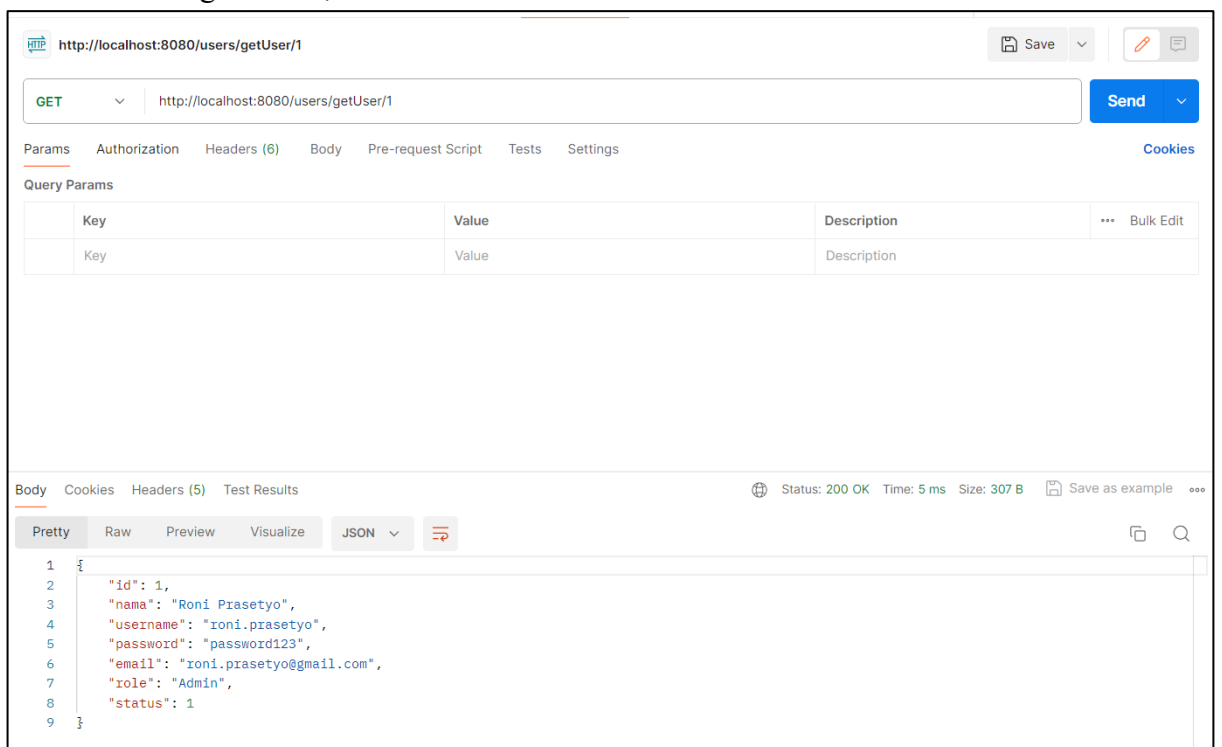
47 @PutMapping("/updateUser/{id}")
48 public String updateUser(@PathVariable int id, @RequestBody User updatedUser) {
49     for (User user : users) {
50         if (user.getId() == id) {
51             user.setUsername(updatedUser.getUsername());
52             user.setPassword(updatedUser.getPassword());
53             user.setEmail(updatedUser.getEmail());
54             return "User updated successfully";
55         }
56     }
57     return "User not found";
58 }

```

19. Coba jalankan kembali, lalu coba kembali di postman



20. Cobalah untuk `getUser/1`, maka data akan berubah



21. Tambahkan juga untuk delete dimana akan menggunakan anotasi `@DeleteMapping`


```

60      @DeleteMapping("/deleteUser/{id}")
61      public String deleteUser(@PathVariable int id) {
62          for (User user : users) {
63              if (user.getId() == id) {
64                  users.remove(user);
65                  return "User deleted successfully";
66              }
67          }
68          return "User not found";
69      }

```

22. Cobalah untuk menggunakan method ini didalam postman, maka akan tampil seperti dibawah ini

The screenshot shows the Postman interface for a DELETE request to `http://localhost:8080/users/deleteUser/1`. The request is successful with a status of 200 OK. The response body, displayed in the 'Body' tab, is 'User deleted successfully'.

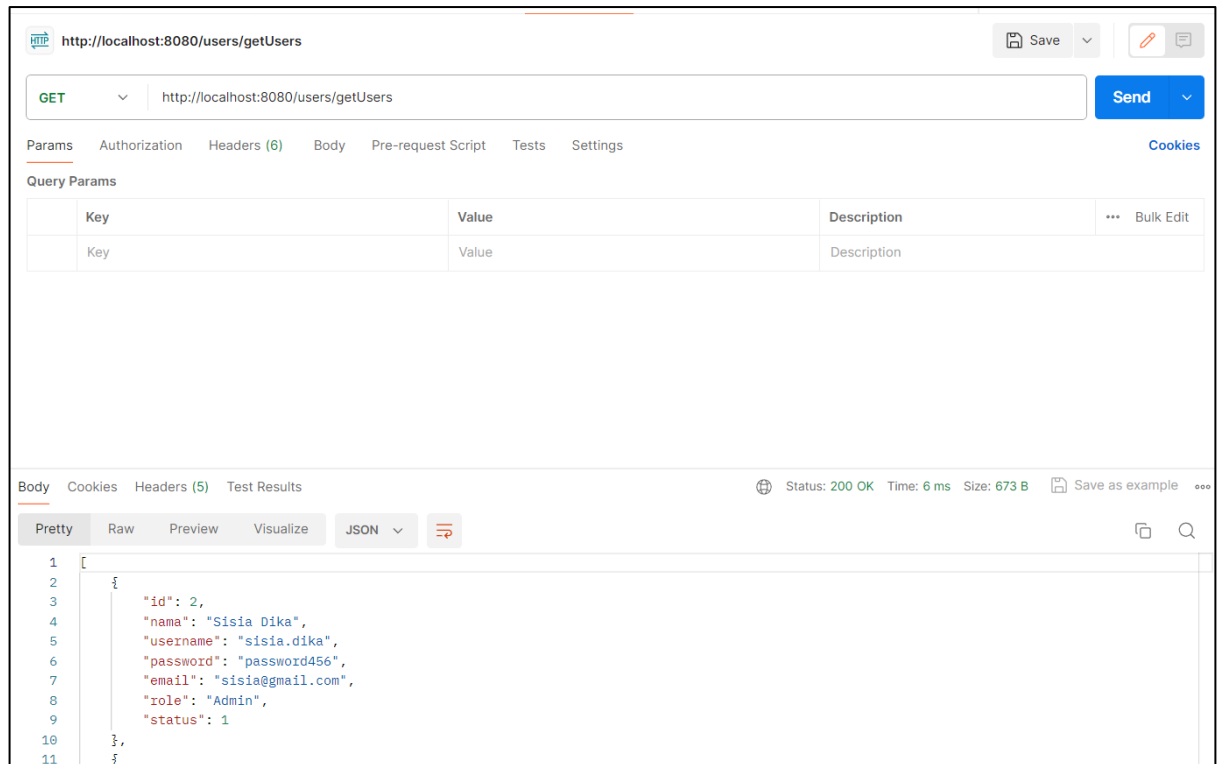
Key	Value	Description
Key	Value	Description

Body: Cookies Headers (5) Test Results

Status: 200 OK Time: 3 ms Size: 189 B Save as example

1 User deleted successfully

23. Selanjutnya, cek didalam getUser/1, maka data yang dikembalikan akan kosong



24. Selanjutnya, buatlah package dengan nama constant, kita akan membuat standarisasi pesan disini, selain itu, nantinya seluruh query native akan disimpan disini. Buatlah kelas java dengan nama UserConstant

```

3 public class UserConstant {
4     2 usages
5     public static final String mNotFound = "User not found";
6     1 usage
7     public static final String mCreateSuccess = "User created successfully";
8     1 usage
9     public static final String mUpdateSuccess = "User updated successfully";
10    1 usage
11    public static final String mDeleteSuccess = "User deleted successfully";
12 }

```

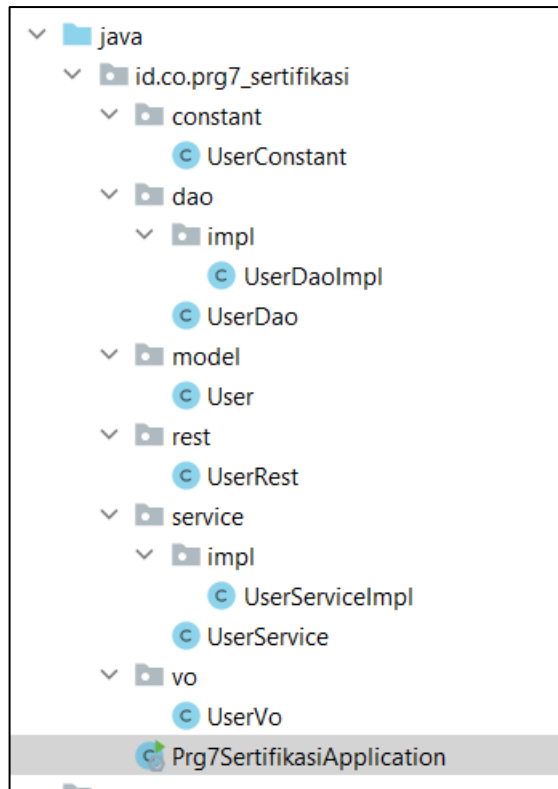
25. Ubahlah pesan yang berada di Rest menjadi menggunakan variabel yang telah terstandarisasi

```

29 @PostMapping("/saveUser")
30 public String createUser(@RequestBody User user) {
31     int nextId = users.size() + 1;
32     user.setId(nextId);
33     users.add(user);
34     return mCreateSuccess;
35 }

```

26. Selanjutnya, kita akan bekerja menggunakan vo, service, dan dao. Buatlah kelas dan package secara lengkap seperti dibawah ini



27. Disini, kita akan mencoba memulai dari vo, View Object adalah representasi objek dari data yang ditampilkan dalam antarmuka pengguna (UI). Disini kita hanya akan menampilkan id, nama, username, email, dan role

```

1  package id.co.prg7_sertifikasi.vo;
2
3  import id.co.prg7_sertifikasi.model.User;
4
5  public class UserVo {
6      3 usages
7      private int id;
8      3 usages
9      private String nama;
10     3 usages
11     private String username;
12     3 usages
13     private String email;
14     3 usages
15     private String role;
16
17     public UserVo() {
18     }
19
20     @ public UserVo(User user) {
21         this.id = user.getId();
22         this.nama = user.getNama();
23         this.username = user.getUsername();
24         this.email = user.getEmail();
25         this.role = user.getRole();
26     }
27
28     public int getId() {
29         return id;
30     }

```

28. Selanjutnya, pindahkan users ke dalam constant sehingga menjadi seperti dibawah ini

```

1 package id.co.prg7_sertifikasi.constant;
2
3 import id.co.prg7_sertifikasi.model.User;
4
5 import java.util.ArrayList;
6 import java.util.List;
7
8 public class UserConstant {
9     public static List<User> users;
10
11     static {
12         users = new ArrayList<>();
13         users.add(new User( id: 1, nama: "Roni Prasetyo", username: "roni.p", password: "password123", email: "roni@gmail.com", role: "Admin"));
14         users.add(new User( id: 2, nama: "Sisia Dika", username: "sisia.dika", password: "password456", email: "sisia@gmail.com", role: "Admin"));
15         users.add(new User( id: 3, nama: "Ning Ratwastuti", username: "ning.r", password: "password789", email: "ning@gmail.com", role: "Admin"));
16         users.add(new User( id: 4, nama: "Elora Manuella", username: "elora.a", password: "password101", email: "elora@gmail.com", role: "Admin"));
17     }
18
19     public static final String mNotFound = "User not found";
20     public static final String mCreateSuccess = "User created successfully";
21     public static final String mUpdateSuccess = "User updated successfully";
22     public static final String mDeleteSuccess = "User deleted successfully";
23 }

```

29. Selanjutnya, kita akan memulai dari UserDao, bukalah UserDao, buatlah kelas ini menjadi kelas interface yang nantinya akan diimplementasikan di UserDaoImpl

```

1 package id.co.prg7_sertifikasi.dao;
2
3 import id.co.prg7_sertifikasi.model.User;
4 import id.co.prg7_sertifikasi.vo.UserVo;
5
6 import java.util.List;
7
8 public interface UserDao {
9     UserVo getUserById(int id);
10    List<UserVo> getAllUsers();
11    void saveUser(User user);
12    void updateUser(User user);
13    void deleteUser(int id);
14 }

```

30. Bukalah UserDaoImpl, disini kita akan mengimplementasi method method yang telah kita deklarasi

```

11  @Repository
12  public class UserDaoImpl implements UserDao {
13
14      1 usage
15      @Override
16      public UserVo getUserById(int id) {
17          for (User user : UserConstant.users) {
18              if (user.getId() == id) {
19                  UserVo userVo = new UserVo(user);
20                  return userVo;
21              }
22          }
23          return null;
24      }
25
26      1 usage
27      @Override
28      public void saveUser(User user) {
29          UserConstant.users.add(user);
30      }
31
32      1 usage
33      @Override
34      public void updateUser(User user) {
35          for (User existingUser : UserConstant.users) {
36              if (existingUser.getId() == user.getId()) {
37                  existingUser.setNama(user.getNama());
38                  existingUser.setUsername(user.getUsername());
39                  existingUser.setPassword(user.getPassword());
40                  existingUser.setEmail(user.getEmail());
41                  existingUser.setRole(user.getRole());
42                  existingUser.setStatus(user.getStatus());
43                  return;
44              }
45          }
46      }
47
48      1 usage
49      @Override
50      public void deleteUser(int id) {
51          UserConstant.users.removeIf(user -> user.getId() == id);
52      }
53
54      1 usage
55      @Override
56      public List<UserVo> getAllUsers() {
57          List<UserVo> userVos = new ArrayList<>();
58          for (User user : UserConstant.users) {
59              UserVo userVo = new UserVo(user);
60              userVos.add(userVo);
61          }
62          return userVos;
63      }
64  }

```

31. Disini kita menggunakan anotasi `@Repository` yang berarti adalah bagian dari lapisan akses data dalam arsitektur aplikasi. Selain itu, kita mengimplement kelas dari `UserDao`.

32. Selanjutnya, bukalah UserService dan ketikan kode seperti dibawah ini

```
1 package id.co.prg7_sertifikasi.service;
2
3 import id.co.prg7_sertifikasi.model.User;
4 import id.co.prg7_sertifikasi.vo.UserVo;
5
6 import java.util.List;
7
8 5 usages 1 implementation
9 3 usages 1 implementation public interface UserService {
10 1 usage 1 implementation UserVo getUserById(int id);
11 1 usage 1 implementation void saveUser(User user);
12 1 usage 1 implementation void updateUser(User user);
13 1 usage 1 implementation void deleteUser(int id);
14 1 usage 1 implementation List<UserVo> getUsers();
15 }
```

33. Seperti kelas dao, kelas service ini akan berbentuk interface

34. Selanjutnya, dibagian UserServiceImpl kita akan mengimplementasikan seluruh method dari iterface diatas dengan anotasi @Service. Anotasi ini berarti untuk menandai bahwa kelas tersebut adalah bagian dari lapisan layanan dalam arsitektur aplikasi Spring.

```

1  package id.co.prg7_sertifikasi.service.impl;
2
3  import id.co.prg7_sertifikasi.dao.UserDao;
4  import id.co.prg7_sertifikasi.model.User;
5  import id.co.prg7_sertifikasi.service.UserService;
6  import id.co.prg7_sertifikasi.vo.UserVo;
7  import org.springframework.stereotype.Service;
8
9  import java.util.List;
10
11  @Service
12  public class UserServiceImpl implements UserService {
13      6 usages
14      private final UserDao userDao;
15
16      public UserServiceImpl(UserDao userDao) {
17          this.userDao = userDao;
18      }
19
20      3 usages
21      @Override
22      public UserVo getUserById(int id) {
23          return userDao.getUserById(id);
24      }
25
26      1 usage
27      @Override
28      public void saveUser(User user) {
29          userDao.saveUser(user);
30      }

```

```

31      1 usage
32      @Override
33      public void updateUser(User user) {
34          userDao.updateUser(user);
35      }
36
37      1 usage
38      @Override
39      public void deleteUser(int id) {
40          userDao.deleteUser(id);
41      }
42
43      1 usage
44      @Override
45      public List<UserVo> getUsers() {
46          return userDao.getAllUsers();
47      }
48  }

```

35. Selanjutnya, kita coba modifikasi UserRest, dengan memanggil fungsi dari UserService


```

13  @RestController
14  @RequestMapping("/users")
15  public class UserRest {
16      8 usages
17      private final UserService userService;
18
19      public UserRest(UserService userService) {
20          this.userService = userService;
21      }
22
23      @GetMapping("/getUsers")
24      public ResponseEntity<List<UserVo>> getUsers() {
25          List<UserVo> users = userService.getUsers();
26          return new ResponseEntity<>(users, HttpStatus.OK);
27      }
28
29      @PostMapping("/saveUser")
30      public ResponseEntity<String> createUser(@RequestBody User user) {
31          userService.saveUser(user);
32          return new ResponseEntity<>(mCreateSuccess, HttpStatus.CREATED);
33      }
34
35      @GetMapping("/getUser/{id}")
36      public ResponseEntity<UserVo> getUserById(@PathVariable int id) {
37          UserVo user = userService.getUserById(id);
38          if (user != null) {
39              return new ResponseEntity<>(user, HttpStatus.OK);
40          } else {
41              return new ResponseEntity<>(HttpStatus.NOT_FOUND);
42          }
43      }
44
45      @PutMapping("/updateUser/{id}")
46      public ResponseEntity<String> updateUser(@PathVariable int id, @RequestBody User updatedUser) {
47          UserVo existingUser = userService.getUserById(id);
48          if (existingUser != null) {
49              updatedUser.setId(id);
50              userService.updateUser(updatedUser);
51              return new ResponseEntity<>(mUpdateSuccess, HttpStatus.OK);
52          } else {
53              return new ResponseEntity<>(mNotFound, HttpStatus.NOT_FOUND);
54          }
55      }
56
57      @DeleteMapping("/deleteUser/{id}")
58      public ResponseEntity<String> deleteUser(@PathVariable int id) {
59          UserVo existingUser = userService.getUserById(id);
60          if (existingUser != null) {
61              userService.deleteUser(id);
62              return new ResponseEntity<>(mDeleteSuccess, HttpStatus.OK);
63          } else {
64              return new ResponseEntity<>(mNotFound, HttpStatus.NOT_FOUND);
65          }
66      }
67  }

```

36. Coba jalankan satu per satu fungsi yang ada, maka disini kita telah bekerja dengan menggunakan beberapa package sesuai fungsi yang ada di Spring, mulai dari constant, model, vo, dao, service, dan rest.

5. LATIHAN

1. Tambahkan fungsi untuk mencari user berdasarkan username dengan API `/users/getByUsername`.
2. Tambahkan validasi pada saat menambahkan data user apabila username dan email telah digunakan akan memberikan pesan "Username or Email has been registered".
3. Tambahkan fungsi CRUD Prodi dengan detail sebagai berikut

Kolom	Tipe Data
Id	Integer
Prodi	String
Singkatan	String
Jurusan	String
Status	Integer